

GROUP 61 | ASSIGNMENT 02

CollabBoard

Working REST APIs with In-Memory Data

Implementation, integration, verification, and submission evidence

Technical status: Frontend and REST implementation passed automated and live integration QA. Final Git tagging remains gated on human approval and genuine commits from all five members.

NSBM Green University

Group 61

31 August 2026

1. Introduction

CollabBoard is a responsive Kanban-style collaboration workspace for small project teams. Assignment 02 connects the completed React frontend to a modular Express REST API while preserving the frontend workflows, data field names, and derived selector logic established during the frontend campaign.

The implementation uses deterministic server-side in-memory data, JWT authentication, bcrypt password hashes, explicit CORS, allowlisted validation, stable service facades, and an executable Postman collection. MongoDB, realtime features, deployment, Docker, offline synchronization, and richer collaboration features are intentionally outside this assignment boundary.

Assignment outcome: The React application now performs authentication, membership-scoped project and task workflows, collaboration-aware Team reads, and Settings updates through protected REST endpoints without direct page-level fetch calls or automatic mock fallback.

2. Team Members and Roles

Student ID	Member	Assignment role
35762	M N R Mudannayaka	Frontend, backend integration, and project coordination
35783	M D J Prabashana	Frontend components and task API verification
33223	K I S S Kariyawasm	UI/UX, responsive design, and integration verification
35934	D M S P Bandara	Dashboard, supporting UI, API verification, and QA
30824	C R Gammbheera	Common components, Postman, documentation, and Git

Contribution evidence must reflect work each person actually performed. At the time this report was generated, Git history did not yet contain verified individual commits from all five members, so the final Assignment 02 tag remains deliberately pending.

3. System Architecture



Figure 1. Assignment 02 frontend-to-backend request flow.

The stable frontend service layer is the architectural seam. Pages depend on authService, projectService, taskService, and userService. Each facade selects either the explicit mock adapter or the REST adapter. In API mode, apiClient owns JSON serialization, the Bearer header, 204 handling, normalized ApiError objects, and 401 session clearing.

- WorkspaceProvider loads users, projects, and tasks concurrently through service contracts.
- Express routes delegate to validation-aware controllers and an in-memory repository.
- Shared modules contain only public enums and public seed entities.
- Password hashes, JWT configuration, and authentication records remain backend-only.

4. Frontend-Backend Integration

The migration keeps existing root service imports stable, so page components did not need a second redesign. The REST adapters return the same entity shapes as the asynchronous mock adapters. After successful mutations, WorkspaceProvider refetches canonical collections, ensuring the UI reflects server state rather than optimistic local-only changes.

Frontend operation	HTTP request	UI result
authService.login	POST /api/auth/login	JWT session and protected dashboard
projectService.create	POST /api/projects	New project after canonical refresh
taskService.updateStatus	PATCH /api/tasks/:id/status	Task appears in the new Kanban column
userService.update	PATCH /api/users/:id	Shared current-user profile updates

5. REST API Design

The API is a small monolithic Express application organized by routes, controllers, services, repositories, middleware, data, and utilities. Success responses use bare entities or arrays. Deletions return 204 No Content. Errors use a single envelope with code, message, and details fields.

Error contract: `{"error":{"code":"TASK_NOT_FOUND","message":"Task not found","details":[]}}`

Correct HTTP semantics are used: 400 invalid input or immutable-field change, 401 missing or invalid JWT, 403 visible owner-only operation, non-disclosing 404 for inaccessible entities, 409 identity or member-assignment conflict, and 500 unexpected server errors.

6. Authentication

Seeded development users authenticate with the password password. The server stores only bcrypt hashes in memory. Successful login and registration return a JWT and a public user. Tokens contain sub, iat, and exp and expire after eight hours by default.

1. The Login page calls `authService.login` with email, password, and Remember Me preference.
2. The REST auth adapter posts credentials to `/api/auth/login` and stores the returned token in `localStorage` or `sessionStorage`.
3. `apiClient` adds `Authorization: Bearer <token>` to protected requests.
4. The Express middleware validates the token and attaches the public user to the request.
5. A 401 clears the browser session and returns the application to Login.

Task `creatorId` and project `ownerId` are derived from the JWT. Profile updates are self-only. Projects are visible only to their owner and members; project management is owner-only, while project members may perform task CRUD inside accessible projects. Task `projectId` is immutable after creation.

7. API Endpoint Summary

Group	Public	Protected operations
Health	GET <code>/api/health</code>	None
Authentication	POST <code>/api/auth/register</code> ; POST <code>/api/auth/login</code>	GET <code>/api/auth/me</code>
Users	None	List, read, self-profile patch
Projects	None	List, read, create, patch, delete
Tasks	None	List/filter, read, create, patch, status patch, delete

Task queries first restrict results to accessible non-archived projects and then accept projectId, assigneeId, status, priority, and label. Progress, workload, overdue totals, Dashboard counts, and project contribution remain frontend selector responsibilities. GET /api/users remains the registered account directory; the Team UI separately derives only shared-project collaborators.

8. GitHub Repository and Assignment 02 Tag

Repository: <https://github.com/RamsaraNohan/CollabBoard>

Frontend base commit: 4ac9e849c85dff5cca5a1c349771cf16b549b1a0

Implementation branch: codex/assignment02-rest-api

Planned final tag: assignment-02-working-rest-apis

Tag status: Not created. The approved process requires all five genuine member contributions and human approval of the final implementation commit before the annotated tag is created and pushed.

9. How to Run

1. Run npm install in the repository root.
2. Run npm --prefix server install for the separate backend package.
3. Copy server/.env.example to server/.env and set a local JWT_SECRET of at least 16 characters.
4. Run npm run dev:all.
5. Open <http://localhost:5173> and sign in with member1@example.com / password.

Frontend-only mock development remains explicit through npm run dev:mock. Complete commands, environment variables, testing scripts, API boundaries, and credentials are documented in README.md.

10. Postman Collection

The Postman v2.1 collection is stored at postman/CollabBoard_Assignment02.postman_collection.json. It defines baseUrl, ownerToken, userBToken, and deterministic entity variables. Its access-scoping sequence registers an unassigned user, verifies empty visibility, grants and revokes membership, checks non-disclosing 404 responses, validates member-removal conflicts, and restores seeded state.

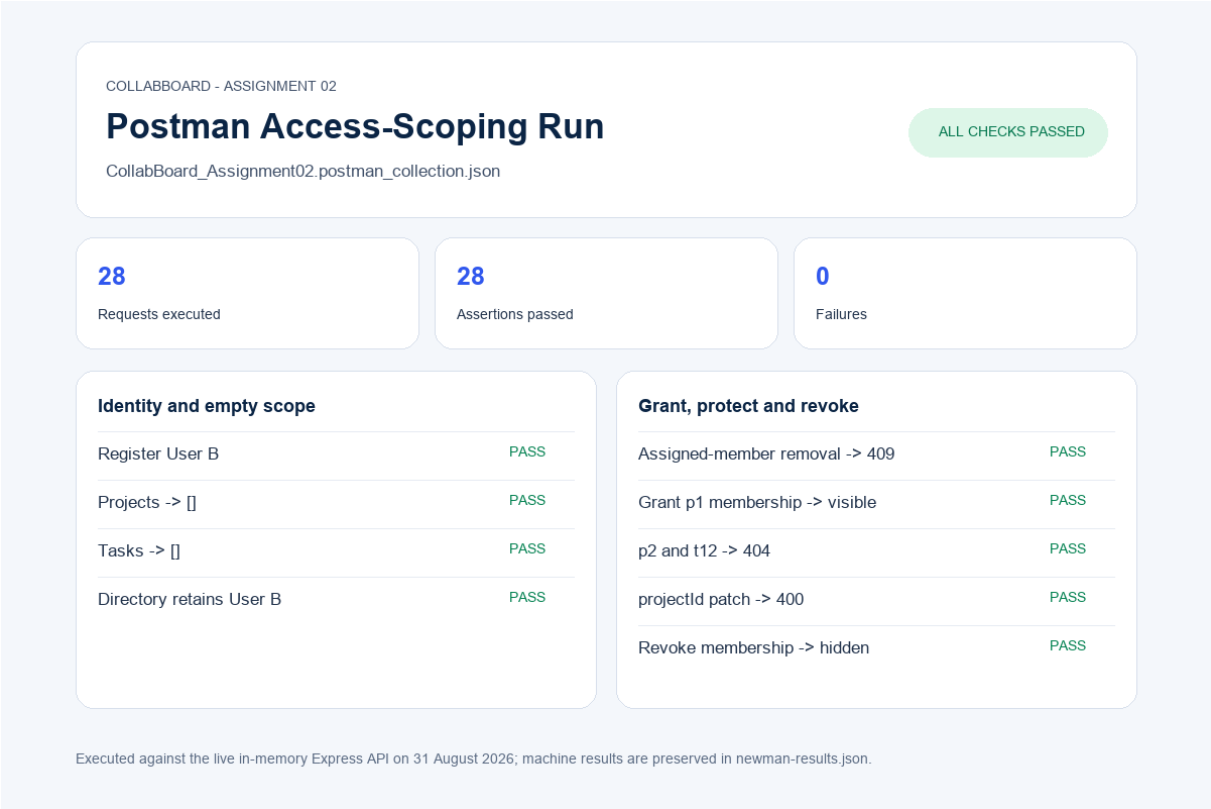


Figure 2. Executable access-scoping collection result: 28 requests, 28 assertions, 0 failures.

11. Frontend Screenshots

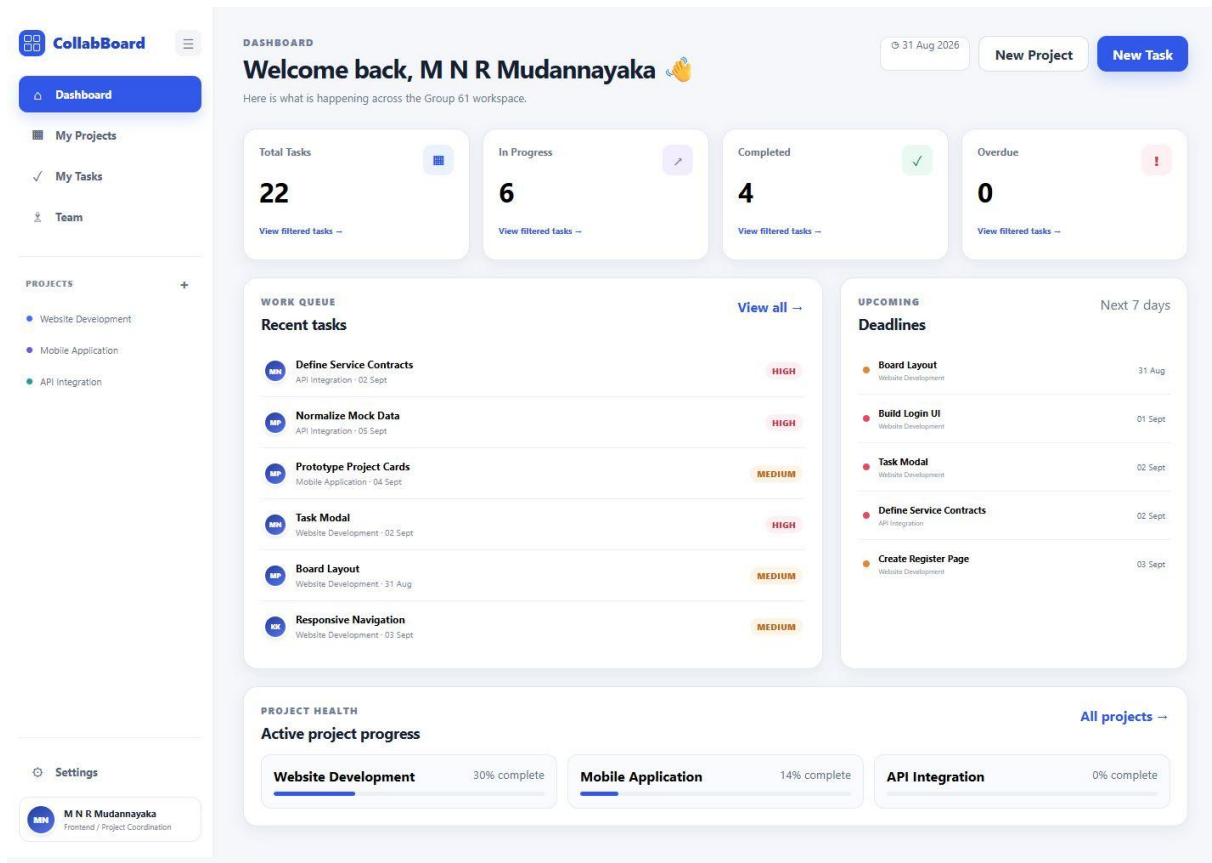


Figure 3. API-backed Dashboard with derived task counts and deadlines.

The screenshot displays the 'My Projects' section of the CollabBoard application. The left sidebar contains navigation links: Dashboard, My Projects (selected), My Tasks, Team, and Settings. Below these is a 'PROJECTS' section with a list of projects: Website Development, Mobile Application, and API Integration. The main content area, titled 'WORKSPACES My Projects', shows three project cards. Each card includes a title, status, description, task progress, and team members.

Project Name	Status	Description	Tasks	Progress	Team	Due Date
Website Development	ACTIVE	Group #1 full-stack workshop project.	10 tasks: 0 overdue	30% complete	MN, MP, DS, CC	Due 21 Sept
Mobile Application	ACTIVE	Responsive mobile experience planning and prototype work.	7 tasks: 0 overdue	14% complete	MN, MP, DS, DB	Due 30 Sept
API Integration	PLANNING	Frontend service contracts and Assignment 02 integration preparation.	5 tasks: 0 overdue	0% complete	MN, MP, DS, CC	Due 08 Oct

Figure 4. Canonical project list loaded from the REST API.

CollabBoard

MY PROJECTS / WEBSITE DEVELOPMENT

Website Development

Overview • Board • Members

Search tasks, labels, assignees...

All priorities All assignees All labels

To Do 4

- Build Login UI**
Create a responsive login page with validation states.
HIGH 01 Sept
M N R Mudannayaka
- Create Register Page**
Design and implement the user registration interface.
MEDIUM 03 Sept
M D J Prabhana
- Wireframe**
Create low-fidelity wireframes for the main application screens.
LOW 04 Sept
K I S S Kariyawam
- Component Tree**
Define the React component hierarchy for the application.
MEDIUM 05 Sept
C R Gammheera

Doing 3

- Board Layout**
Implement the main Kanban board layout and responsive sidebar.
MEDIUM 31 Aug
M D J Prabhana
- Task Modal**
Create a reusable Add and Edit Task modal component.
HIGH 02 Sept
M N R Mudannayaka
- Responsive Navigation**
Adapt desktop navigation for tablet and mobile screens.
MEDIUM 03 Sept
K I S S Kariyawam

Done 3

- React Project Setup**
Scaffold the Vite and React application structure.
LOW 28 Aug
M N R Mudannayaka
- Create GitHub Repository**
Create the team repository and agree on branch strategy.
MEDIUM 28 Aug
C R Gammheera
- Design Dashboard**
Create task summary cards and recent activity sections.
LOW 29 Aug
D M S P Bandara

Dashboard

My Projects

My Tasks

Team

PROJECTS

- Website Development
- Mobile Application
- API Integration

Settings

M N R Mudan...
Frontend / Project ...

Figure 5. Project-specific Website Development Kanban board.

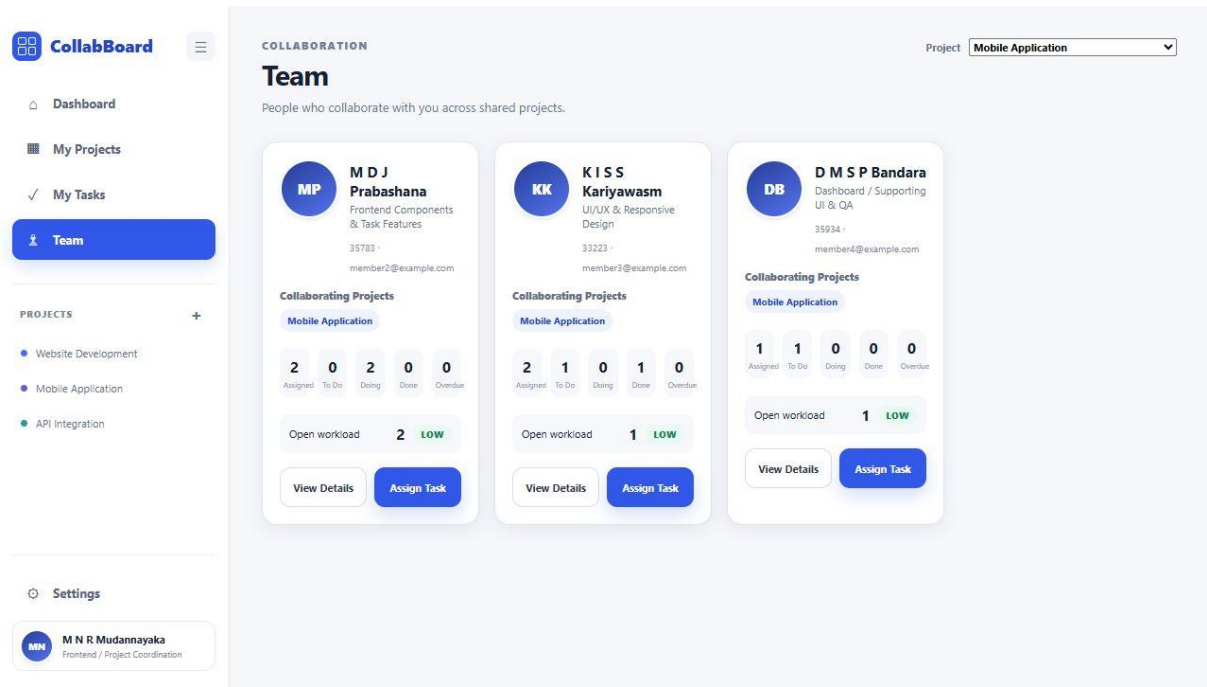


Figure 6. Shared-project Team collaborators and project-scoped metrics.

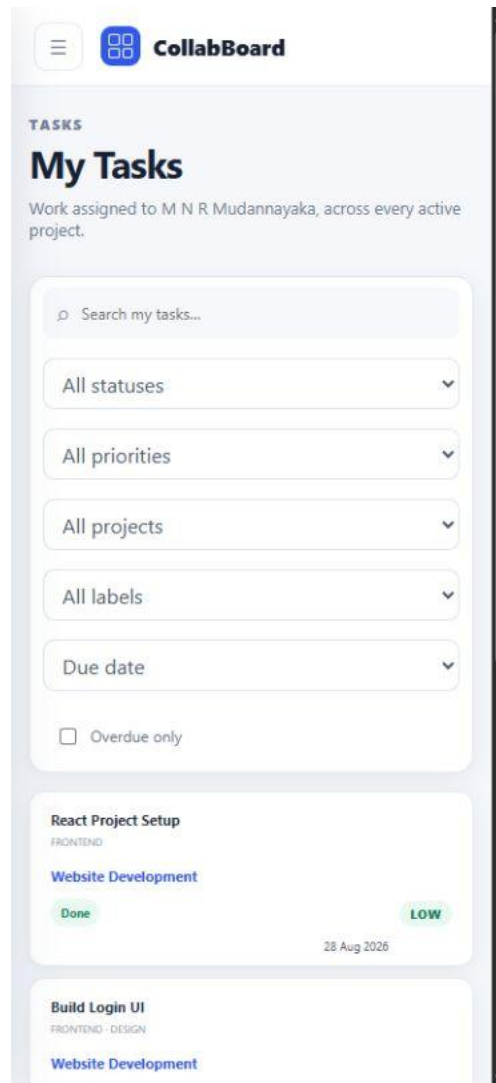


Figure 7. Mobile My Tasks card layout at 390 px.

12. Backend/API Screenshots

The screenshot displays the 'Express API Verification' interface. At the top, it says 'COLLABBOARD - ASSIGNMENT 02' and 'Express API Verification' with a 'RUNNING' status. Below this, three summary boxes show: '200 GET /api/health', '14/14 Backend tests', and '28/28 Postman assertions'. A dark blue box titled 'Runtime and access-policy evidence' contains terminal logs showing successful health checks, 401 responses for unauthorized access, and 403 responses for unauthorized modifications. At the bottom, a note states: 'JWT secrets remain local and ignored. Network Retry and real-401 session behavior were verified in the React client.'

COLLABBOARD - ASSIGNMENT 02

Express API Verification

Membership-scoped JWT REST API with deterministic in-memory data

RUNNING

200
GET /api/health

14/14
Backend tests

28/28
Postman assertions

Runtime and access-policy evidence

```
$ npm run dev:server
CollabBoard API listening on http://localhost:5000/api
GET /api/health -> 200 { status: ok, service: collabboard-api }
GET /api/projects (no JWT) -> 401
GET /api/projects (new user) -> []
GET /api/projects/p1 (non-member) -> 404
PATCH /api/projects/p1 (visible non-owner) -> 403
PATCH /api/tasks/t1 { projectId: p3 } -> 400 IMMUTABLE_FIELD
Test Files 1 passed | Tests 14 passed
```

JWT secrets remain local and ignored. Network Retry and real-401 session behavior were verified in the React client.

Figure 8. Express startup, health response, protected-route 401, and test summary.

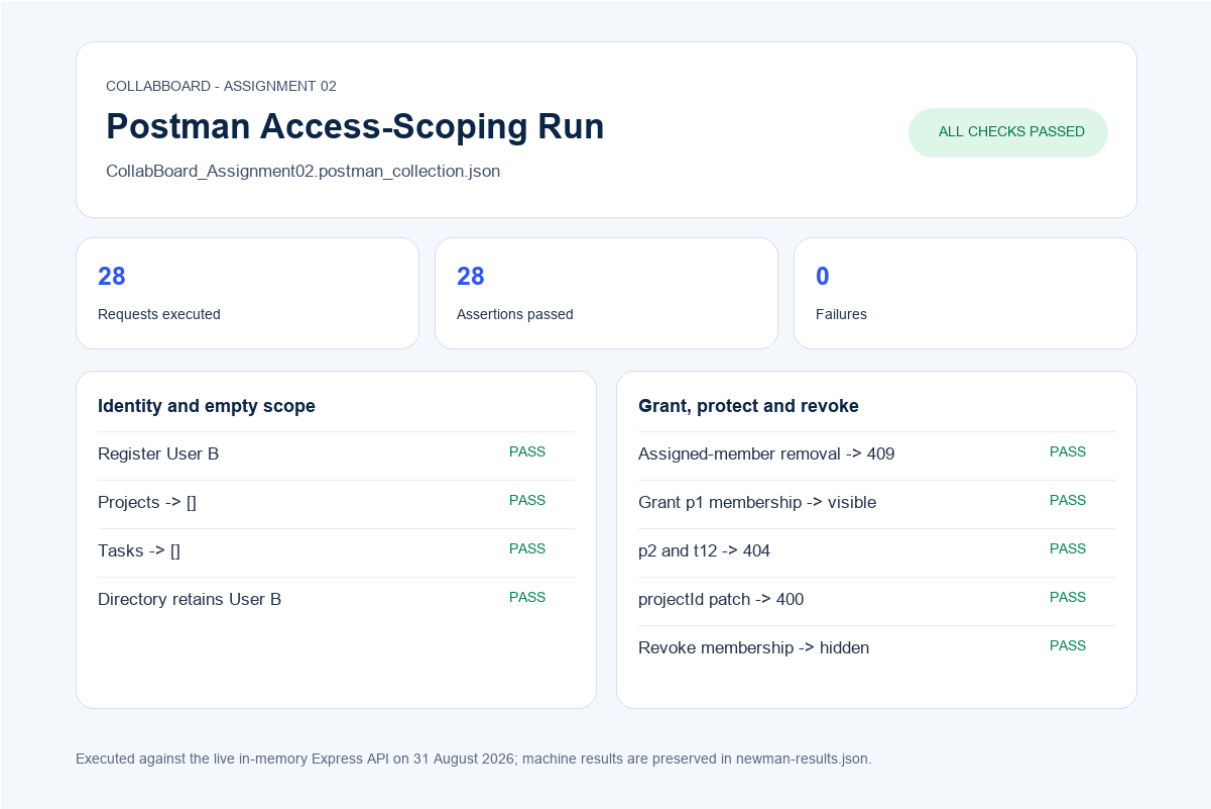


Figure 9. Postman/Newman request and assertion summary.

13. Testing and Verification

Gate	Result	Evidence
Frontend tests	PASS	9 files, 20 tests
Backend tests	PASS	1 file, 14 tests
Postman/Newman	PASS	28 requests, 28 assertions, 0 failures
Production build	PASS	Vite transformed 104 modules
Responsive browser QA	PASS	390, 768, 1024, and 1440 px
Console audit	PASS	Offline, Retry, 401, and 409 flows had no unhandled rejection

Live browser QA used real API mode. It verified startup, login and JWT restoration, offline error handling and Retry, invalid-token clearing, zero-project empty states, project-scoped Sidebar data, Team union and project filters, assignment project choice, owner-only controls, direct route refreshes, and responsive layouts.

Backend tests additionally verified user-scoped project and task lists, archive semantics, access grant and revocation, the member-removal 409 sequence, non-disclosing entity reads, member task CRUD, invalid assignees, immutable projectId, self-only user updates, and normalized 400/401/403/404/409 errors.

14. Known Limitations

- In-memory mutations reset whenever the Express server restarts.
- The JWT design intentionally omits refresh tokens and account recovery for this coursework milestone.
- MongoDB, realtime updates, offline queues, comments, notifications, attachments, and deployment are deferred.
- The final Git tag is pending human approval and verified genuine contributions from all five members.
- Registered users are not automatically added to existing projects.
- Archived projects remain directly readable to members but have no archive-center UI in this campaign.

15. Conclusion

CollabBoard now satisfies the technical Assignment 02 objective: a completed React frontend communicates through stable asynchronous service contracts with a JWT-protected Express REST API

using deterministic in-memory backend data. Project and task visibility is enforced by ownership and membership on the server, while Team collaboration is derived only from shared active projects and presentation metrics remain frontend-owned.

The implementation has passed automated frontend and backend tests, an executable Postman collection, production build, live integration checks, responsive browser QA, and visual evidence review. Submission tagging must wait for the remaining people-dependent contribution and approval gates; no artificial commits or premature tag are included.